ALGOPROFESSOR AI R&D SOLUTIONS · HYDERABAD

AlgoLang 1.1

The agent-native programming language for trustworthy AI



Dr. S. Satyanarayana, Ph.D., PDF (AI)

Founder, CEO & Chief Agentic AI Scientist · AlgoProfessor AI R&D Solutions · www.algoprofessor.com

github.com/drssnaiml1-cell/algolang · Apache 2.0 · November 2026

Agenda

01 The problem: AI decisions nobody can check
02 What AlgoLang is, and its design principles
03 The language: agents, flows, policies, types
04 Compiler, runtime and the AlgoVM
05 Governance and Trustworthy AI
06 Uncertainty, evals and evidence
07 Real-time events, Quantum AI, interoperability
08 Comparison with programming languages
09 Comparison with agent frameworks
10 22 enterprise applications
11 Toolchain, roadmap, getting started
12 Where AlgoLang fits, and collaboration

The problem

AI agents now make decisions that matter, and nobody can check them

- Loans, triage, claims, admissions and trades are increasingly decided by LLM agents.
- In today's code an agent is just a function call buried in Python: **no policy, no budget, no reviewer, no record**.
- Regulation now asks for risk classes, human oversight, transparency and logs (e.g. the EU AI Act, India's DPDP Act).
- Teams bolt these on afterwards, inconsistently, and often not at all.

Four questions every AI decision should answer

1. Who was allowed to decide?
2. Why was this the decision?
3. Was it fair and robust?
4. Can an auditor prove what happened?

AlgoLang makes the compiler and runtime answer them, every time.

What is AlgoLang?

AlgoLang is a programming language with **no functions**. Every unit of work is an **agent** with a role, a goal, explicit tools, a model and a policy.

Agent

role · goal · tools · model · policy; returns text, score, list, a typed message, or `uncertain<T>`

Flow

the only other construct: call agents, run them in parallel, branch, emit results

Guarantees

policies, budgets, human review and a signed audit log enforced by compiler and runtime

Designed by Dr. S. Satyanarayana at AlgoProfessor AI R&D Solutions, Hyderabad. Open source (Apache 2.0).

Design principles

- **Agents, not functions.** The unit of computation is an accountable agent.
- **Policies are code.** Budgets, forbidden attributes and reviewers are declared, and checked before anything runs.
- **Uncertainty is a type.** `uncertain<score>` samples several times and carries its confidence.
- **Deterministic guards.** Branch on tool verdicts, not model moods.
- **People stay in charge.** `require human`, four-eyes approvals, escalation queues.
- **Evidence by default.** A hash-chained, signed audit record for every call.
- **Tested like software.** Evals for quality, fairness and robustness in CI.
- **Offline first.** A deterministic mock model, so every program runs and tests without an API key.

The language at a glance

A governed loan decision in 20 lines

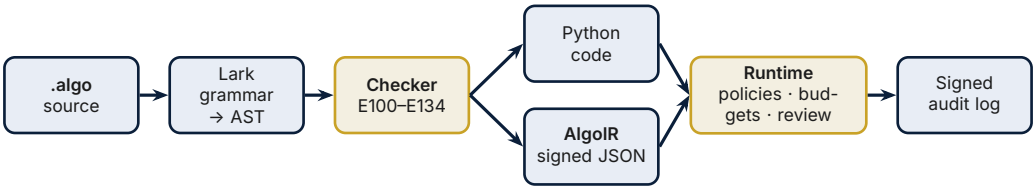
```
risk high "Retail credit decisions"
disclose "An AI system assisted this decision."

agent Affordability(app: text) -> score {
    goal   "Debt-to-income ratio for {app}"
    uses   bureau.pull
    policy { budget tokens 2000  redact pii }
}
agent Committee(app: text, dti: score) -> uncertain<score> {
    goal   "Recommend approval for {app}"
    policy { samples 5  require approvals 2
             forbid gender, religion  explain with evidence }
}
flow main(app: text) {
    dti = Affordability(app)
    if dti > 0.5 { emit Decline(app, dti) }
    else         { emit Committee(app, dti) }
}
```

- `risk high`: the compiler demands oversight, fairness tests, PII redaction and signed audit.
- `budget`, `forbid`: refused at compile time if violated.
- `uncertain<score>`: 5 samples, mean and confidence.
- `require approvals 2`: four-eyes review before release.
- Every call → signed, hash-chained audit.

Compiler and runtime

From source to governed execution



35 compile-time checks: undeclared tools, budgets, forbidden attributes, type errors, unsafe high-risk programs.

Two engines, same result: compiled Python or the AlgoVM; 223 programs identical in CI.

Backends: Claude, OpenAI-compatible local models, offline mock, replay from traces.

Governance built into the language

Control	What it does	Syntax
Token, cost, energy budgets	checked before every call; INR and USD; Wh and CO_2	`budget cost 0.04 usd`
Forbidden attributes	a protected attribute in a goal or prompt does not compile	`forbid gender, caste`
Human review	by rule, by confidence, or always; a persistent review queue	`require human when confidence < 0.8`
Four-eyes approval	two different people must approve	`require approvals 2`
Tool permissions	only declared tools; untrusted tools are fenced	`tool web.fetch untrusted`
Organisation policy	org-wide caps that no program can loosen	`algo.config.json`
Risk levels	minimal · limited · high, each with required safeguards	`risk high "purpose"`

Trustworthy AI (v0.4)

Trustworthiness as evidence, not a claim

- **Risk levels** that change what the compiler demands.
- **Privacy**: `redact pii`; protected values withheld even with typos.
- **Explanations**: `explain with evidence` quotes inputs and tool results.
- **Untrusted tools** fenced against prompt injection; MCP servers pinned by SHA-256.
- **Signed audit**: Ed25519, and since v0.6 ML-DSA (FIPS 204) or hybrid post-quantum signatures.
- **Retention**: audit and trace retention with signed checkpoints.
- **System cards** and **trust reports** generated from the program itself.
- **Evidence packs**: signed zip for auditors, verifiable offline.

Uncertainty and evals

Quality, fairness and robustness tested like code

```
eval trust on main {
    case "salaried" (app = "APP-3101 ...") {
        expect status ok
        expect fair across gender [woman, man]
        expect robust (typos, paraphrase, reorder)
               consistency >= 0.9
        expect explained
    }
}
```

```
$ algo eval programs/loan_decision.algo
✓ salaried applicant
  fair across gender: same decision, gap 0.000
  robust to typos, paraphrase, reorder:
  consistency 1.00 over 6 variants
✓ self-employed applicant
  2/2 cases pass
```

`uncertain<T>` samples an agent several times; flows branch on `.confidence`; low confidence goes to a person.

Typed agent-to-agent messages (v1.0)

```
type Party { name: text, phone_verified: bool }
type Claim { policy: text, amount: number,
             fraud_risk: score, claimant: Party }

agent Intake(note: text) -> Claim {
    goal "Fill in every field of the claim in {note}"
}
flow main(note: text) {
    claim = Intake(note)
    if claim.amount < 50000 and claim.fraud_risk < 0.7 {
        emit FastTrack(claim)
    } else {
        emit Investigate(claim, claim.amount)
    }
}
```

- Fields and argument types checked by the compiler (E131, E132).
- Every model answer validated against the type at run time.
- The live model is told the exact JSON shape.
- PII masked field by field.

New in 1.1: accountable decisions

Three questions every governed AI decision raises, answered by the program itself

Is every answer within its limits?

Agent contracts `ensure`

Checked on every answer. Broken → one more try → a person. The audit record says which contracts held.

Would the decision have been different?

What-if audits `algo whatif`

Re-runs a recorded decision with one input changed; unaffected agents are replayed, so the comparison is exact.

Did this change weaken a safeguard?

Governance diff `algo diff`

Marks every policy, tool, contract and eval change as loosened, tightened or changed. Fails CI when loosened.

```
agent Payout(claim: Claim, cover: number) -> number {
    goal   "Fair payout for {claim}"
    ensure result >= 0     else "never negative"
    ensure result <= cover else "never above cover"
}
```

```
$ algo whatif run.trace.json --set cover=5000
  2 of 3 calls re-computed
  ✗ the decision changed
$ algo diff git:main:claims.algo claims.algo
  ▼ loosened Payout contract removed
  ▼ loosened Payout 'gender' allowed again
```

Beyond the core

Real-time events, Quantum AI, interoperability and the ecosystem

AlgoPulse™

Event streams in the language: topics, keys, dedupe, escalation and dead-letter topics, back-pressure.

Quantum AI

Simulator, noisy simulators, IBM, Braket and Azure backends; QPU budgets; VERIFIED / BOUNDED answers; VQE, kernels, QAOA.

Interoperability

Flows and agents as MCP servers; typed inputs; token streaming; cost and energy dashboard.

Ecosystem

AlgoIR + AlgoVM, signed packages and registry, Studio with SSO and roles, VS Code, Python SDK.

Quantum hardware backends are exercised in CI with stand-in SDKs; they need the provider's credentials to run on a real device.

No quantum speed-up is claimed: quantum answers are checked before use.

Comparison with general-purpose languages

What each language is designed for

Language	Paradigm	Typing	Concurrency	Strength
AlgoLang	agent-oriented, declarative flows	static: text, score, list, json, number, typed messages	`par`, `par for`, streams	governed AI decisions: policies, review, audit, evals
Python	multi-paradigm	dynamic (optional hints)	threads, asyncio, processes	ecosystem for AI, data and scripting
Java	object-oriented	static	threads, virtual threads	large enterprise systems on the JVM
C / C++	procedural / multi-paradigm	static	threads	performance, systems, embedded
JavaScript / TS	multi-paradigm	dynamic / static (TS)	event loop, async/await	web and full-stack applications
Rust	multi-paradigm, ownership	static	threads, async; data-race freedom	memory safety without a garbage collector
Go	procedural, CSP	static	goroutines and channels	cloud services and tooling

AlgoLang is a domain-specific language: it orchestrates AI agents and delegates computation to tools written in general-purpose languages.

Comparison with scientific, logic and AI-specific languages

Language	Focus	Core idea	Relation to AlgoLang
AlgoLang	governed AI agents	agents with policies; uncertainty and evals as language features	—
R	statistics	vectorised data analysis, rich statistical libraries	tools for analysis inside agents
Julia	scientific computing	multiple dispatch, JIT compilation	numerical tools inside agents
MATLAB	engineering maths	matrices, toolboxes	engineering tools inside agents
SQL	data querying	declarative relational queries	data access tools for agents
Prolog	logic programming	facts, rules, unification	symbolic rules beside model agents
Lisp	symbolic AI	code as data, macros	historic AI language; no governance layer
LMQL	constrained LLM prompting	queries with constraints on model output	prompt-level; AlgoLang is program-level
Q# / Qiskit	quantum programming	quantum circuits and algorithms	AlgoLang calls circuits as verified tools

Comparison with agent frameworks

Libraries on a host language vs. a language with guarantees

Framework	Form	Approach	Governance
AlgoLang	language + compiler	agents and flows as syntax; checked before running	built in: compile-time and run-time
LangChain / LangGraph	Python, JS library	chains and stateful agent graphs	added by the developer in code
CrewAI	Python library	role-based crews of agents	added by the developer in code
AutoGen	Python, .NET library	multi-agent conversations	added by the developer in code
Semantic Kernel	C#, Python, Java SDK	plugins and planners in enterprise apps	added by the developer in code
DSPy	Python library	declarative LM programs, optimised prompts	added by the developer in code

Frameworks are powerful and mature, and have far larger communities. AlgoLang's difference is **where** the guarantees live: in a compiler that refuses an unsafe program, instead of in conventions each team must remember to follow.

Feature matrix

✓ built in ◐ via libraries or conventions ✗ not available

Capability	AlgoLang	Python	Java	Rust	TS	Julia	Lang-Graph	CrewAI
Agents as a language construct	✓	✗	✗	✗	✗	✗	◐	◐
Budgets checked before running	✓	✗	✗	✗	✗	✗	◐	◐
Human review / four-eyes	✓	◐	◐	◐	◐	◐	◐	◐
Uncertainty as a type	✓	◐	◐	◐	◐	◐	✗	✗
Fairness and robustness evals	✓	◐	◐	✗	◐	◐	◐	◐
Signed, hash-chained audit	✓	◐	◐	◐	◐	◐	◐	◐
Typed agent messages	✓	◐	◐	◐	◐	◐	◐	◐
Quantum tools, verified	✓	◐	◐	◐	◐	◐	✗	✗
General-purpose computing	✗	✓	✓	✓	✓	✓	◐	◐
Mature ecosystem and community	✗	✓	✓	✓	✓	✓	✓	✓
Raw performance	◐	◐	✓	✓	◐	✓	◐	◐

The same guarantee, two ways

A budgeted agent with human review on low confidence

Python with a framework (sketch)

```python
def triage(claim):
    if tokens_used() > 2000:
        raise BudgetError()
    out = llm.invoke(prompt(claim))
    score, conf = parse(out)
    log_audit(claim, out)        # remember to sign
    if conf < 0.8:
        queue_for_review(out)    # remember to persist
    if "gender" in prompt(claim):
        ...                      # remember to check
    return score
```

AlgoLang

```
agent Triage(claim: text) -> uncertain<score> {
    goal   "Score {claim}"
    policy { budget tokens 2000
             require human when confidence < 0.8
             forbid gender }
}
```

Budget, review queue, forbidden attribute and signed audit are **enforced**, not remembered.

Where AlgoLang is not the right tool

An honest view

- **General computation.** No loops over numbers, no data structures beyond its types: write those as tools in Python or another language.
- **High-performance code.** The runtime is Python; speed comes from tools.
- **A young ecosystem.** One maintainer team, a small community and few third-party packages so far.

Use AlgoLang when

- AI agents make decisions that affect people or money
- you must show regulators who decided and why
- fairness, budgets and human review are requirements, not wishes
- you want agent programs that are reviewed and tested like code

22 governed enterprise applications

Each with policies, company tools, knowledge base, evals, behaviour tests and system cards

Enterprise

- Insurance claims operations
- Hospital care coordination
- Bank credit and KYC
- University admissions and OBE
- SOC incident command
- Supply chain control tower
- Citizen services
- Legal contract desk

AI, markets and quantum

- Algo trading desk
- Quantum optimization lab
- AlgoQ Quantum AI lab
- AlgoGraph LLM & SLM optimizer
- ML model factory
- RL control room
- Responsible XAI studio
- NLP document intelligence

Large-scale domains (v0.5)

- Bharat language desk (8 scripts)
- Digital health monitor (NEWS2)
- Network biology lab
- Predictive maintenance
- Smart city operations
- Transport network optimizer

22	38	223	35
applications	programs	programs on AlgoVM	compile checks

Applications · Enterprise operations

enterprise-apps/claims-ops-center, hospital-care-coordination

01 / 22

LIMITED RISK

Claims Operations Center

Insurance

Challenge First notice of loss takes hours: policy checks, fraud screening and a fair payout estimate.

WHAT IT DOES

- Reads the claim and pulls cover, deductible and status from the policy database
- A three-member fraud panel runs in parallel
- High fraud score → SIU referral note, and the flow stops

Payout drafts carry evidence; fraud referrals stop automation

`policydb.lookup payout.estimate` 2 programs

02 / 22

HIGH RISK

Hospital Care Coordination

Healthcare

Challenge Emergency departments must triage fast, find a bed, hand over cleanly and write discharge summaries.

WHAT IT DOES

- Extracts key facts from the ER note into a structured summary
- Scores urgency; urgent cases get a live bed and a referral draft
- Drafts discharge summaries for clinician sign-off

Every score below threshold goes to the duty clinician

`file.read beds.availability` 2 programs

Applications · Enterprise operations

enterprise-apps/bank-credit-onboarding, university-admissions-obe

03 / 22

HIGH RISK

Bank Credit & Onboarding

Banking

Challenge Loan and KYC decisions need bureau data, affordability maths and sanctions screening, all explainable.

WHAT IT DOES

- Pulls the bureau record and computes EMI and debt-to-income
- Credit, risk and policy analysts review in parallel
- Credit committee scores the case and writes a decision memo

Gender and religion forbidden; low confidence → a person

`bureau.score dti.calc sanctions.screen`

2 programs

04 / 22

HIGH RISK

University Admissions & OBE

Higher education

Challenge Admissions must be merit-based and fair, and every course needs NBA/NAAC outcome-based evidence.

WHAT IT DOES

- Checks seats and merit cut-offs for the programme
- Academic, statement and potential reviewers assess in parallel
- Panel scores the case and drafts the letter; CO-PO attainment for OBE

Caste, gender, religion, nationality, income: forbidden inputs

`seats.lookup file.read`

2 programs

Applications · Enterprise operations

enterprise-apps/soc-incident-command, supply-chain-control-tower

05 / 22

LIMITED RISK

SOC Incident Command

Cybersecurity & IT operations

Challenge Security operations centres drown in alerts; analysts need assets, owners and threat intel at once.

WHAT IT DOES

- Enriches alerts from the CMDB and checks every IP and domain against threat intel
- Splits the batch and scores severity in parallel
- Severe → IT operations runbook; post-quantum readiness scan

Untrusted intel fenced against prompt injection

`cmdb.lookup ioc.check pqc.scan` 3 programs

06 / 22

MINIMAL RISK

Supply Chain Control Tower

Logistics & retail

Challenge Customs holds, breakdowns and damage need a quick view of where stock is and what to do next.

WHAT IT DOES

- Tracks the shipment and checks stock cover in parallel
- Scores how critical the exception is
- Critical → impact, alternatives, costs and a recovery plan

Quantum route options verified before they are proposed

`shipment.track inventory.levels quantum.routes` 2 programs

Applications · Enterprise operations

enterprise-apps/citizen-services-desk, legal-contract-desk

07 / 22

HIGH RISK

Citizen Services Desk

Government & public services

Challenge Complaints must reach the right department fast; urgent ones (health, safety, children) first.

WHAT IT DOES

- Finds ward, department, desk and service-level target
- Urgent → priority cell with an action plan
- Checks every reply for clarity; welfare eligibility pre-check

AI disclosure on every reply; eligibility decided by people

`ward.directory`

2 programs

08 / 22

MINIMAL RISK

Legal Contract Desk

Legal & compliance

Challenge In-house legal teams review dozens of NDAs and contracts a week against the same playbook.

WHAT IT DOES

- Extracts NDA terms and scores deviation from the playbook
- Suggests redlines with approved fallback wording
- Analyses every contract clause in parallel and summarises for counsel

Counsel decides; the desk drafts with cited clauses

`clause.library file.read`

2 programs

Applications · AI, markets & quantum

enterprise-apps/algo-trading-desk, quantum-optimization-lab

09 / 22 MINIMAL RISK

Algo Trading Desk (QuantOracle™)

Capital markets

Challenge A trading desk must decide which strategies are robust enough to trade, within risk and conduct rules.

WHAT IT DOES

- Quant, compliance and editorial reviewers run in parallel
- Robustness is an `uncertain<score>` sampled five times
- Failing backtests (PF < 1.3 or DD > 15%) are rejected by a guard

Approval only for PAPER trading, through the review queue

`backtest.stats risk.limits market.quote` 4 programs

10 / 22 MINIMAL RISK

Quantum Optimization Lab

Quantum for finance & operations

Challenge Many business decisions are combinatorial: which k of n assets to hold, which jobs go on which shift.

WHAT IT DOES

- Portfolio selection as a QUBO, solved with QAOA
- Max-cut, scheduling and certified quantum randomness
- Every answer checked against the exact classical optimum

Guard on VERIFIED / GAP; max qubits enforced by policy

`quantum.portfolio quantum.qubo quantum.maxcut` 4 programs

Applications · AI, markets & quantum

enterprise-apps/quantum-ai-lab, llm-graph-optimizer

11 / 22

MINIMAL RISK

AlgoQ Quantum AI Lab™

Quantum computing

Challenge Quantum computers are noisy, scarce and expensive, and their answers are distributions, not facts.

WHAT IT DOES

- VQE ground-state energies, checked by exact diagonalisation
- Knapsack capital budgeting and spectrum colouring with QAOA
- Quantum kernels cross-validated against a classical RBF

QPU budgets; only VERIFIED answers reach a two-approval committee

`quantum.vqe quantum.kernel quantum.knapsack` 1 program

12 / 22

LIMITED RISK

AlgoGraph™ LLM & SLM Optimizer

AI platform engineering · MLOps

Challenge Large models are slow and costly; small models are cheap but not always good enough.

WHAT IT DOES

- Reads each program's agent graph from the compiler
- Picks the cheapest LLM or SLM per agent that meets quality and latency
- Proves the choice optimal; routes live traffic small-model-first

Route decided in code, not by a model

`graph.analyze graph.optimize slm.triage` 2 programs

Applications · AI, markets & quantum

enterprise-apps/ml-model-factory, rl-control-room

13 / 22

LIMITED RISK

AlgoML Model Factory™

Machine learning & deep learning

Challenge Shipping a model safely needs more than a high score: fair comparison, calibration and drift checks.

WHAT IT DOES

- Trains three model families in parallel with 5-fold cross-validation
- Picks a calibrated champion (ECE $\leq$ 0.05)
- Catches target leakage before training; monitors drift

A leaking column stops the run before any training

`ml.train ml.compare ml.drift`

1 program

14 / 22

MINIMAL RISK

AlgoRL Control Room™

Reinforcement learning for energy

Challenge An RL controller can beat a hand-written rule, but a grid cannot afford trial-and-error mistakes.

WHAT IT DOES

- Q-learning for a 2 MWh campus battery over 3,000 simulated days
- Safety shield enforces the 1.75 MW grid-import cap
- Benchmarked against the exact optimum, then heatwave stress tests

Policy deploys only after it beats the rule and stays safe

`rl.train rl.evaluate rl.stress`

1 program

Applications · AI, markets & quantum

enterprise-apps/responsible-xai-studio, nlp-document-intelligence

15 / 22 LIMITED RISK

AlgoXAI Audit Studio™

Responsible & explainable AI

Challenge A credit model can be accurate and still unfair, often through a feature that stands in for a protected one.

WHAT IT DOES

- Exact Shapley values over all 2^6 coalitions
- Fairness audit on a group used only for auditing
- Actionable recourse: the smallest change that flips a decline

Age and area are never suggested as changes

`xai.shapley fair.audit xai.counterfactual` 1 program

16 / 22 LIMITED RISK

AlgoNLP Document Intelligence™

Natural language processing

Challenge A bank receives thousands of messages a week: card blocks, loans, KYC updates, fraud reports.

WHAT IT DOES

- Classifies, extracts entities and summarises every message in parallel
- Emerging topics and distinctive words per category
- Reply drafts routed to the right queue

Fraud reports always go to a person, with evidence

`nlp.classify nlp.entities nlp.trends` 1 program

Applications · Large-scale domains

enterprise-apps/bharat-language-desk, digital-health-monitor

17 / 22 LIMITED RISK

AlgoBharat Bhasha Desk™

Indian-language AI for citizen services

Challenge Citizens write in their own language and script: Telugu, Hindi, Tamil, Bengali and more.

WHAT IT DOES

- Detects the language from 11 scripts; separates Hindi from Marathi
- Transliterates Brahmic scripts (ISO 15919 style)
- Finds the need (pension, ration, water…) and replies in kind

Coverage reported honestly; unknown scripts go to a person

`lang.detect lang.transliterate lang.intent` 1 program

18 / 22 HIGH RISK

AlgoCare Remote Monitoring™

Healthcare & digital health

Challenge Thousands of patients are monitored at home; their readings must be triaged safely, around the clock.

WHAT IT DOES

- Scores every reading with NEWS2, exactly by published bands
- Checks the record and major drug interactions in parallel
- High → emergency alert; medium → review within the hour

A clinician reviews every alert

`vitals.news2 ehr.patient drug.check` 1 program

Applications · Large-scale domains

enterprise-apps/network-biology-lab, predictive-engineering

19 / 22 MINIMAL RISK

AlgoBio Network Lab™

Computational & network biology

Challenge Disease genes cluster in the protein network, so known proteins can point to new candidates.

WHAT IT DOES

- Analyses a 1,500-protein, 7,441-interaction network in parallel
- Random walk with restart, validated on hidden disease proteins
- Negative control from random seeds; enrichment and sequences

- A weak control is required before results are reported

`net.propagate net.modules bio.enrich` 1 program

20 / 22 MINIMAL RISK

AlgoEng Predictive Maintenance™

Data-driven engineering

Challenge Unplanned pump failures stop production and can be dangerous; the data shows them coming.

WHAT IT DOES

- Vibration diagnosis by ISO zones, in parallel with prognosis
- Remaining useful life with a 90% interval
- Fleet scan and a maintenance plan

- Urgent work orders are approved by a person

`vib.diagnose rul.estimate maint.plan` 1 program

Applications · Large-scale domains

enterprise-apps/smart-city-operations, transport-network-optimizer

21 / 22 LIMITED RISK

AlgoCity Operations Centre™

Smart cities

Challenge A city control room sees air, waste and electricity data around the clock, each in its own silo.

WHAT IT DOES

- Checks AQI (CPCB), waste routes and energy load in parallel
- Very poor AQI → multilingual health advisory
- Overload risk → load plan; never hospitals, water or signals

Advisories need the duty officer; load plans need two officers

`city.aqi city.waste city.energy` 1 program

22 / 22 LIMITED RISK

AlgoMove Transport Network Optimizer™

Transportation systems

Challenge When an arterial road closes, traffic floods side streets within minutes; fixed signals cannot cope.

WHAT IT DOES

- Reroutes traffic (Dijkstra) and retimes signals (Webster) in parallel
- Gravity-model demand and transit headways
- Oversaturated junctions → police control and metering plan

Control-room officer approves; notices carry AI disclosure

`traffic.reroute signal.retime demand.od` 1 program

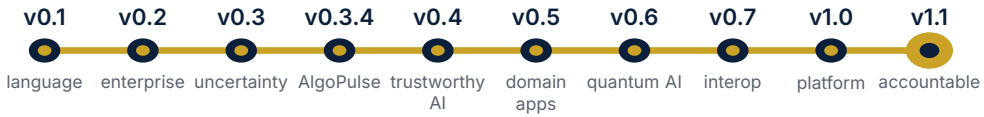
Toolchain

```
$ pip install algolang
$ algo check app.algo
$ algo run app.algo --arg "claim=..."
--stream
$ algo eval app.algo
$ algo card app.algo
$ algo build app.algo --ir --sign
$ algo mcp serve app.algo
$ algo pkg install insurance@^1.0
$ algo studio
```

- **CLI** `algo`: check, run, batch, eval, card, report, audit, evidence, build, pkg, mcp.
- **Studio**: browser IDE with review queue, dashboard, SSO and roles.
- **Python SDK** and Jupyter / Colab.
- **VS Code** extension and language server.
- **CI**: Linux, macOS and Windows on every change.

Roadmap delivered

Every planned release shipped, one at a time



Latest: 1.1 agent contracts · what-if audits · governance diff, on the 1.0 platform.

After 1.1 AlgoVM for browser and edge · organisation registries · federated review · long-running agents.

Getting started in five minutes

```
$ git clone https://github.com/
  drssnaiml1-cell/algolang
$ cd algolang && pip install -e .
$ algo run examples/hello.algo \
  --arg "name=Asha"
$ algo studio
```

Runs offline with the mock model; add `ANTHROPIC_API_KEY` and `--live` for real models.

- **50 sample programs** with a PDF guide.
- **108-page language manual** (PDF).
- **Website**: playground and package registry.
- **Releases** with wheels for every version.

Collaborate with us

- **Researchers and Ph.D. scholars**: agentic AI, XAI, AI governance, quantum AI; joint papers and theses.
- **Universities**: teach governed AI with a language built for it.
- **Industry**: pilot trustworthy agents in a real workflow.
- **Developers**: contribute to an open-source language.

Open research questions

- Fairness guarantees at compile time
- Verifying quantum answers before agents act
- Auditable AI decisions in health and finance
- Typed messages and multi-agent reliability

Build your application with AlgoLang

Governed AI for your sector, from pilot to production

- **Pilot**: a governed agent in one of your real workflows, with evals and an evidence pack.
- **Custom applications**: a sector application like the 22 shown here, built on your tools and policies.
- **Training**: teach teams and students to build and audit trustworthy agents.
- **Research**: joint papers and theses on agentic AI, XAI, AI governance and quantum AI.

Applications & partnerships

cto@algoprofessor.com

Leadership ceo@algoprofessor.com
Web www.algoprofessor.com
Code github.com/drssnaiml1-cell/algolang

AlgoProfessor AI R&D Solutions, Hyderabad, India

Thank you

Build AI people can trust, then prove it.

Dr. S. Satyanarayana, Ph.D., PDF (AI)

Founder, CEO & Chief Agentic AI Scientist · AlgoProfessor AI R&D Solutions, Hyderabad

Applications: cto@algoprofessor.com · github.com/drssnaiml1-cell/algolang

AlgoLang is open source under the Apache 2.0 licence.